Break Before You Shape

Exact line breaking from a precompiled, certified width model

Raouf Rahiche · Version 1.0 · 2026-11-05 · github.com/Rahiche/break-before-you-shape
Every number in this paper is read from result files produced on one Apple M3 machine and on GitHub Actions.

Abstract. Text engines shape a paragraph before they can break it into lines, because widths depend on kerning, ligatures and contextual forms. We show that, for the fonts and scripts we test, a line's shaped width is a *local* function of its text, and that this locality can be *proven* from a font's layout tables. We compile a shaper's behaviour into a small per-font model by measuring it as a black box, and a static analysis of OpenType and AAT tables certifies the lines on which the model is exact. Lines are then broken before anything is shaped. Across 449 local font faces, 5,342,060 certified lines (96% of all) contain no error. Inside Flutter, a 12 KB (gzipped) file built once on the developer's machine reproduces Flutter's own line breaks and paragraph heights in 2,524,906 of 2,524,906 paragraph layouts on macOS (nine corpora in six languages, six font sizes, twelve widths), and in all 5,526,948 it decides on Linux, macOS, an iOS simulator and an Android emulator in continuous integration. The same holds with letter and word spacing, height multipliers, `maxLines` and hard line breaks (4,575,628 layouts), for Hebrew, mixed-direction, Arabic, Japanese, Chinese and Korean text in four more fonts (914,316 layouts), and in 15 further checks (2,498,638 layouts) covering struts, font features, rich text, bundled fallback fonts, combining marks, pointed Hebrew, vowelled and letter-spaced Arabic, emoji, and Indic scripts. Text a sidecar does not cover but the device's own fonts do (emoji, Brahmic scripts) is measured once per run through Flutter, which is exact because a change of font is a change of shaping run. Not one layout differs. The breaker hands 0.87% of Latin-script layouts back to the engine and computes paragraph heights 10–14× faster than `TextPainter`. Because the breaker is plain Dart and the sidecar plain data, it also runs where Flutter's own layout cannot, in a background isolate and on a server without Flutter, and on Flutter web, with the same answers; fitting text to a box, re-measuring after a text-scale change and re-paginating at a new size get 36–40× faster (§6).

1 · Introduction

Every mainstream text stack (Chrome, WebKit, Android, Skia's SkParagraph, CoreText, DirectWrite) runs the same order: *shape* the paragraph with a shaper such as HarfBuzz, *break* it into lines using the shaped advances, and often *reshape* around the breaks. Shaping is the expensive step, and much of it is wasted. A chat list computing thousands of item heights, or an editor re-wrapping on every resize, shapes text that it never draws. Figure 1 shows the alternative this paper makes exact: break first, from a precompiled model, then shape only what is drawn.

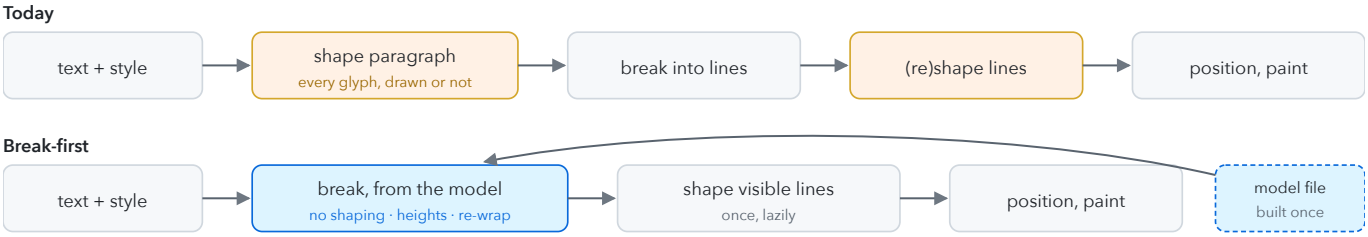


Figure 1. Shape-first layout shapes every glyph before it knows where lines end. Break-first computes line breaks and heights from a precompiled model, so text that is measured but never drawn is never shaped.

Contributions.

1. **Distillation.** A shaper is measured as a black box (every character, every pair, every ligature island in context) and compiled into exact tables. The model matches the target engine by construction.
2. **A certificate.** A static analysis of OpenType (GSUB, GPOS, GDEF) and AAT (`morx`, `kern`, `kerx`, state machines included) tables proves on which lines the model's width equals the shaped width: Arabic letters per joining form, features the app turns on, and combining marks with the compositions HarfBuzz makes.
3. **Break-first layout.** Width-independent records give every candidate line's width in O(1), so a resize costs one greedy pass. The breaker reproduces the host's measured conventions (spacing, line heights, struts, hard breaks, bidirectional runs, split kerning, shaping runs across fonts) and defers wherever the host's behaviour is uncertain.
4. **Evidence.** A census of 449 font faces, and a Flutter package checked against Flutter's own layout on four platforms and the web, for plain and rich text in Latin, Greek, Cyrillic, Hebrew, Arabic, Chinese, Japanese, Korean and Indic scripts and with emoji, and timed in the cases where apps measure text without drawing it (§6).

2 · Width is local, and can be distilled

A line's shaped width is the sum of its glyph advances plus positioning adjustments. In practice those adjustments are *local*: pair kerning touches two adjacent glyphs, and a ligature replaces a short run of letters. So the width of any line decomposes into a few kinds of

terms: one advance per character, one term per adjacent pair, and a term for each ligature *island* with its two neighbours. Figure 2 shows a real decomposition, measured from Flutter.

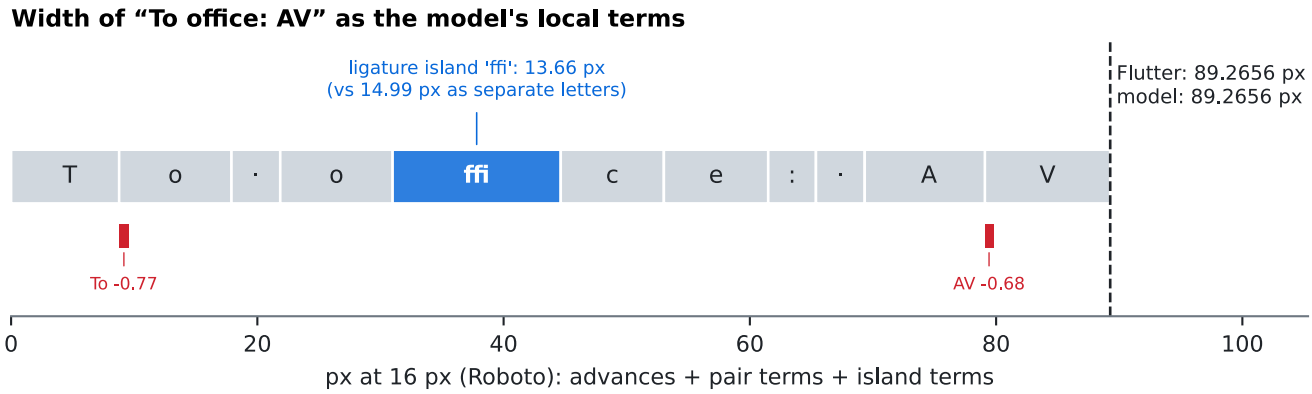


Figure 2. “To office: AV” in Roboto at 16 px. Gray: advances alone, which sum to 92.055 px. Red: pair terms (kerning). Blue: a ligature island. The model sums to 89.2656 px; Flutter measures 89.2656 px.

Distillation. We do not parse kerning or ligature tables. We ask the target shaper. For Flutter, the oracle is Flutter's own `ui.Paragraph`, run under `flutter_tester` on the developer's machine:

- every character alone;
- every ordered pair;
- every certificate island alone, and next to every safe neighbour (§3).

We measure at a font size equal to the font's units per em, so values are font units. All 273,446 measurements came back as exact integers, and widths scale linearly with size (§5.7), so **one file serves every size**. For Roboto (Latin, Greek and Cyrillic, 707 characters), identical rows and columns of the pair matrix collapse into 113×120 classes. With 90 islands, the file is 580 KB, 12.0 KB gzipped, and builds in 4.5 s. The island measurements grow as the island characters to the power m, times the safe characters; when they would pass 2.5 million, the build lowers the bound below the certificate's, and a longer run of island characters goes to Flutter.

---

3 · A certificate from the font's tables

Locality is not guaranteed: a contextual substitution can make a glyph depend on text arbitrarily far away. So each character is classified from the font's layout tables, without running the shaper:

- **S** (safe): no substitution can change or consume its glyph;
- **P** (primer): it can start a substitution, such as the first letter of a ligature;
- **K** (passive): it can only be consumed by a substitution a primer started;
- **U** (unsafe): everything else.

An *island* is a maximal run of P/K characters that contains a primer.

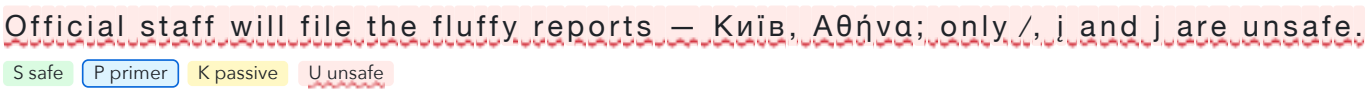


Figure 3. Roboto's certificate on sample text. Islands form only around *f*. Unsafe characters send their paragraph to the engine.

> **Theorem.** Let a line contain no U character, and let every island in it have at most m characters. Then the shaped width of the line equals the model's width.

Proof sketch.

1. **Substitution leaves S glyphs untouched, and islands are closed.**
   - *OpenType.* A single, multiple or alternate lookup fires only at its coverage glyphs, which are U. A contextual rule fires only if every position matches a glyph that can occur, so for each live rule the cheapest position's characters are made U. Ligatures start at P and consume only P/K glyphs.
   - *AAT.* Every `morx` subtable (rearrangement, contextual, ligature, insertion, in either processing order) and the `kerx/kern` state machines are emulated as HarfBuzz runs them. An S class changes nothing from any reachable state and returns to a state equivalent to a start state (state minimisation). Every island up to the bound runs through the whole chain from each start state, with every mark and stack entry from before it stale; if it touches stale state or depends on its surroundings, its cheapest character becomes U.

2. **Positioning is pair-additive.** Only single adjustments, PairPos with ValueFormat2 = 0, the `kern` and `kerx` pair formats, and kerning machines proven local by step 1 can fire.
3. **Width splits into local terms.** By 1 and 2, the width is a sum of terms, each touching at most one island and its S neighbours.
4. **The terms agree.** The model's terms are measured values of exactly those terms. Islands are measured with every S neighbour, and the checker verifies no model term spans two islands. ∎

The proof trusts that the shaper implements OpenType and AAT as specified, that fontTools parses the tables correctly, and the analyzer's code, which is tested but not machine-checked; its AAT emulation was compared with `hb-shape` on about 22,700 islands. By default it also trusts that the shaper selects lookups as HarfBuzz does (the scripts certified, language, default variation coordinates); a strict mode drops that assumption.

Two additions for the breaker. *Pair roles:* for each character, whether its glyph can be the first or second glyph of any pair adjustment (GPOS pair and cursive coverage, `kern` pairs; anything not analysed counts as every glyph). Only first × second pairs are measured, so a CJK font with 28,092 characters needs no pair table at all and one with 16,644 builds in under a second. *Joining forms:* each Arabic (letter, form) is certified as a virtual character whose glyph is the font's isol, init, medi or fina substitute; the breaker picks forms from Unicode joining types, as HarfBuzz's Arabic shaper does, so lam-alef is just an island.

Substitutions and marks. A single substitution that a feature applies to every glyph it covers fixes a character's glyph wherever it stands, so it is part of the character, not a hazard: the analysis follows each character through all of them (small caps, oldstyle figures, stylistic sets). Combining marks get a class of their own, M. HarfBuzz composes a base and a following mark into the precomposed character before any lookup runs, when the font has it, and zeroes the advance of every GDEF mark after positioning. So the certificate lists the font's compositions and, for each mark, its combining class and whether kerning skips it or stops at it (from lookup flags); a rule that can fire only on a mark left uncomposed next to certain characters becomes an exclusion the breaker checks, and a context rule whose nested lookups only move or swap marks cannot change a width at all.

Common characters by script. Spaces, digits and punctuation belong to no script. They take the script of the run they are in, and a run of them alone is shaped with the font's default script, DFLT, whose lookups can differ: Euphemia UCAS's space is 1,030 units under DFLT and 608 in Latin text. So for every other script, the certificate compares the lookups HarfBuzz selects with the main script's, and lists the Common characters and Common pairs that a differing lookup can change. The breaker defers those in a run of that script. When DFLT differs, a string of Common characters is measured before a Latin letter the font lacks: a fallback font draws that letter, so the string shares its script but not its shaping run.

4 · Breaking before shaping, the way the host does

For each paragraph, one pass builds a width prefix in units, and a second finds break opportunities. Both are independent of the box width, so re-wrapping at a new width only re-runs the greedy pass over these records. To reproduce a specific engine, the breaker must also reproduce that engine's conventions. We measured Flutter's rather than assuming them (Table 1), and the breaker **defers** whenever a decision depends on something it cannot certify.

Flutter (SkParagraph) convention	How it was established	Breaker
Shapes once, breaks on shaped cluster advances; kerning sits on the left glyph	Exactness against Flutter's breaks	Pair terms attach to the left character
Line fits if its width, floored to 1/100 px in float32, ≤ box	Accepted and rejected widths around the box edge	Same test, emulated in float32
A word wider than the box is broken inside, on the line where it starts; the "too long" test compares the word's width with the box without the floor, so a word up to 0.01 px wider than the box fits a line of its own yet is broken	2 letter-spaced Greek layouts on GitHub's Linux runner (run 41953508573); 42 constructed words 0.001–0.01 px wider than the box, each broken inside	Defers when the next word is within 1/100 px of the box, or wider
Line height = font line height, rounded to whole px	Every height in Table 2 (2,524,906 on macOS)	lines × rounded height
No kerning across script runs; spaces join the preceding run	"Time Machine" inside Russian text	Pair term 0 at a script change
Spaces, digits and punctuation take their run's script, and a run of them alone is shaped with the font's default script (DFLT), whose lookups can differ	Euphemia UCAS's space: 1,030 units alone, 608 in Latin text	The certificate lists, per script, the characters and pairs whose lookups differ; the breaker defers them there, and the build measures them next to a main-script letter
UAX #14 with Unicode 15.1 number rules; word-initial dash binds	Per-class tallies of taken vs refused breaks	Rule table for the characters it handles
Sometimes breaks at a space inside an unbreakable segment	Examples (e.g. `self ⏎!= 1.`)	Defers when such a space could end the line
Positions drift in float32 at some sizes	Drift by size (§5.7)	Defers decisions the drift could flip

Android truncates the font size to 1/64 px (e.g. 18.4 → 18.390625); macOS and iOS do not	Effective size implied by measured widths at 8 sizes, on the emulator	One probe layout at start-up detects it; sizes are truncated the same way
`\n`, U+2028 and U+2029 end a line; the break kerns with the characters on both sides as a space does on macOS and iOS, not on Linux or Android	"LT↵" loses Roboto's T-space kern on macOS only; with split kerning, "↵A" keeps the A's share (Arial Narrow); CI on Linux	Mandatory records; a start-up probe decides the kerning
Letter spacing is added to every glyph advance (half before the first); positive spacing turns ligatures off, negative keeps them	Widths with spacing against widths with `liga` off	A second certificate without ligatures; spacing per glyph cluster
Word spacing goes on the last space of each run that is a break opportunity, also after a hard break	Glyph positions with and without spacing	Counted per record; defers where a space meets a bidi run boundary
With a height multiplier, a line is round(size × height), size and height taken as float32	15 × 1.3 gives 19, not 20 (heights in Table 4)	The same formula
Kerning from `kern`/`kerx` tables is split between the two glyphs	Times New Roman's space-A kern at a line start	Each pair's right-glyph share, measured from glyph boxes
Bidirectional text: UAX #9 levels; shaping runs split at level changes, and scripts are itemized within each run	Hebrew and Arabic UI strings	Each run measured with the sidecar distilled in its direction
UAX #14 rules LB15c, LB20a, LB21a/b, and LB19a's and LB30's East Asian exceptions; small kana break like ideographs	Validation mismatches, one rule each	Rule table extended
A strut or TextHeightBehavior changes line heights, never line breaks; with one font, all lines but the first and last are alike	Heights in Table 6	Heights from three probe layouts per style
Rich text: a change of font or size starts a shaping run (no kerning or ligature across it), a change of colour or decoration does not; scripts are itemized within each span of one style; a line is round(max ascent + max descent) of its runs	"AV" and "fi" across a colour change; widths across a size change; Euphemia's space next to a letter in another size	Runs per font and size; scripts per style; per-style metrics
Font fallback: each character goes to the first font that has it; scripts are itemized over the whole paragraph, across fonts	A space in Roboto after Chinese text does not kern with the Latin letter after it	Coverage (cmap) in the sidecar; script carried across runs
Combining marks: composed when the font has the composite, zero-width otherwise, kerning skips or stops at them; with positive letter spacing an uncomposed mark is spaced like a glyph	Probes of "e+◌́", "x+◌́", "T◌́o"	Class M, compositions, exclusions; LB9/LB10
Text from the device's fallback fonts is a shaping run of its own	Emoji and Devanagari between Roboto words	Measured once per run through Flutter, cached

Table 1. Host conventions the breaker reproduces, each measured from Flutter.

5 · Evaluation

5.1 · Census: how often does the certificate apply?

We ran the analyzer and the checker on every font face on the machine that covers basic Latin (449 faces). For each, the checker compares every certified line of a 1,000-paragraph English corpus sample at three widths against HarfBuzz, using exact ground truth that reshapes every candidate line.

- **Errors:** 5,342,060 of 5,560,372 lines (96.1%) were certified, and **0 were wrong**.
- **Coverage:** 410 faces certify at least 99% of their lines; 0 certify none.
- **Causes of failure** (39 faces under 99%, Figure 4b): in 13 the font's substitutions genuinely touch common letters; 6 have AAT contextual or ligature machines over common letters that are not local (Apple Chancery, Marion, Aqua Kana); in 0 the exhaustive island check's budget lowered the island bound below 4; the rest lack characters or class letters as marks.

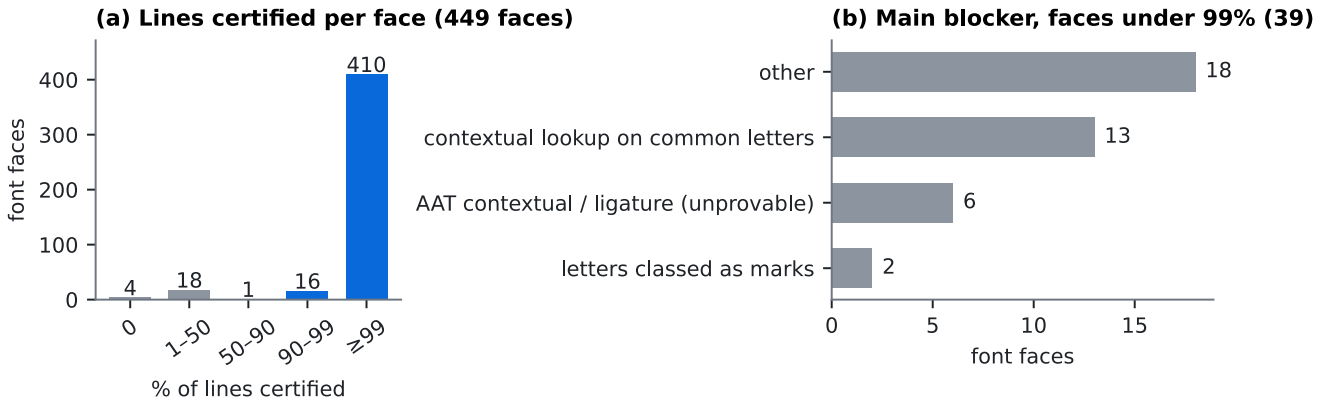


Figure 4. (a) Share of lines certified, per face. (b) The reason behind the most frequent unsafe character, for the faces under 99%.

5.2 · Today's shortcuts are not exact

The usual alternatives to full reshaping lose context, and the error is measurable. Against exact ground truth in HarfBuzz (14 fonts, 5,170 paragraphs, 6 widths; Zapfino is left out, because its long contextual swashes are exactly what the certificate hands back to the shaper):

- **Shape once, then break** (what SkParagraph does) gets up to 1.2% of paragraphs wrong.
- **Per-word caches** (Minikin, Pretext) get up to 9.6% wrong.
- **The distilled model** gets 0.0% wrong.

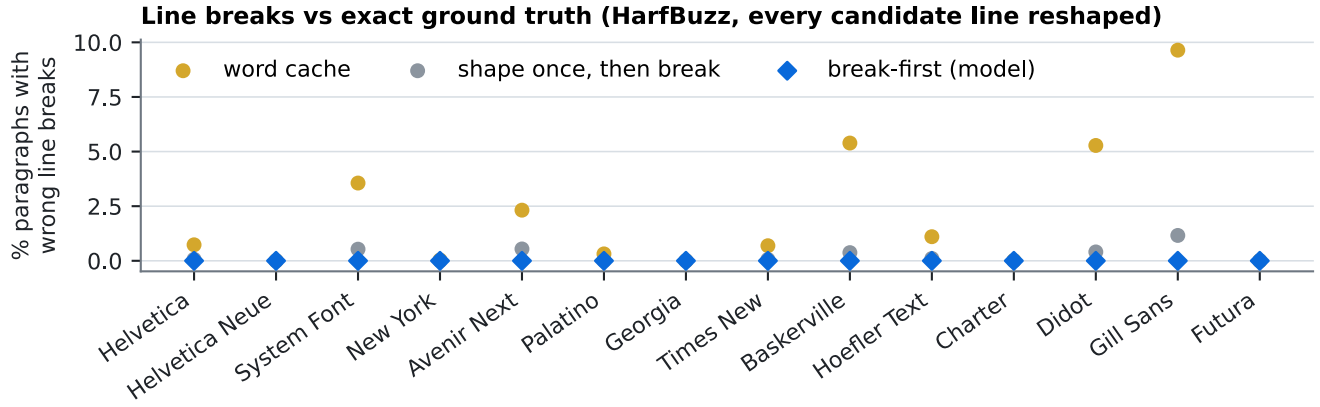


Figure 5. Share of paragraphs whose line breaks differ from exact ground truth (C prototype, HarfBuzz 15).

5.3 · Exactness inside Flutter

The real test is a production engine. A Flutter app (Flutter 3.47.5) loads the sidecar built on the host and compares every line break and every paragraph height with Flutter's own layout. The run covers nine corpora:

- English Python documentation, man pages and license texts;
- user-interface strings in English, German, French, Russian, Ukrainian and Greek, from macOS localizations.

Each paragraph is laid out at 6 font sizes (12, 13, 16, 18.4, 21.3, 24 px) × 12 box widths.

Platform	Paragraph layouts	Identical breaks	Identical heights	Deferred	Canary
macOS, release (all paragraphs)	2,547,000	2,524,906 / 2,524,906	2,524,906 / 2,524,906	0.87%	0.0 px
iOS 17.5 simulator, debug (300 per corpus)	194,184	192,193 / 192,193	192,193 / 192,193	1.03%	0.0 px
Android 15 emulator, release (300 per corpus)	194,184	192,175 / 192,175	192,175 / 192,175	1.03%	0.0 px

Table 2. One sidecar file, built once on macOS, checked against Flutter's layout on three platforms. Lines checked on macOS: 7,201,350.

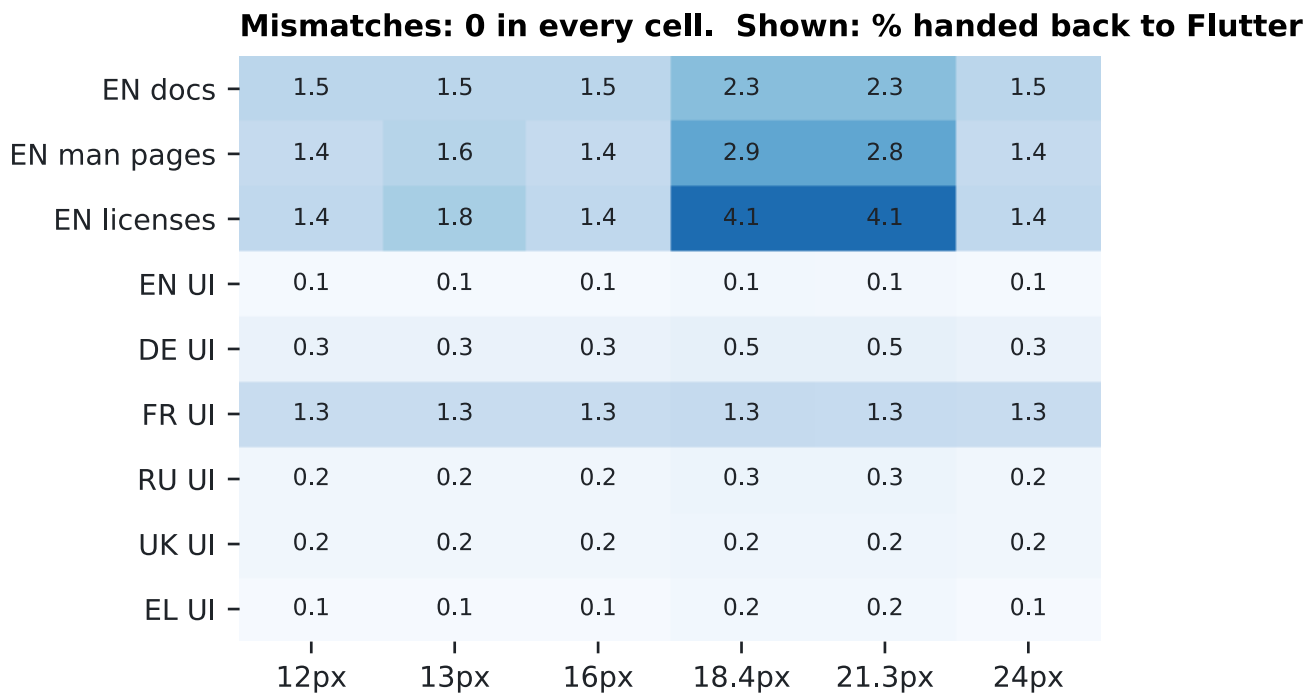


Figure 6. No cell has a single mismatch. Values are the share of paragraph layouts the breaker hands back to Flutter: words wider than the line, characters outside the model, and decisions it will not guess.

On other machines. The same checks run on GitHub Actions (run 41971344191), where each job builds its corpora from its own runner's files. The Linux engine, like Android's, rounds font sizes to 1/64 px; the start-up probe finds it.

Platform (GitHub Actions)	Identical breaks	Different	Heights equal	Size rounding
macOS 15, release (virtual Apple M1)	2,518,525	**0**	2,518,525 / 2,518,525	none
Ubuntu 24.04, Linux desktop release	2,473,065	**0**	2,473,065 / 2,473,065	1/64 px
iOS simulator (CI)	192,168	**0**	192,168 / 192,168	none
Android 15 emulator, x86_64 (CI)	190,348	**0**	190,348 / 190,348	1/64 px
flutter_tester, macOS	76,836	**0**	76,836 / 76,836	none
flutter_tester, Linux	76,006	**0**	76,006 / 76,006	1/64 px

Table 3. The shipped sidecar against Flutter on GitHub's machines, default style.

Every font on the machine. The census of §5.1 checks the model against HarfBuzz; a second census checks the whole package against Flutter. For each of the 447 local faces that cover basic Latin, the certificate is made with the current analyzer, the sidecar is built through Flutter, and 540 paragraphs from the corpora are laid out at 3 sizes × 6 widths, then joined with hard breaks and laid out with letter and word spacing at 4 widths (447 faces; 0 have no certificate or a pair table over the build's budget). Of 5,310,360 layouts, the breaker decides 3,869,528: **3,869,528 identical to Flutter, 0 different**, and every height equal. This census is what found the last rules in Table 1 (Common characters by script, a hard break's kerning on both sides, scripts per style) and two bugs: a ligature skipped when another ligature made the same glyph ("(c)" in Superclarendon), and 8-bit island numbers in a face with more than 128 island characters.

Style options and hard breaks. Each rule in Table 1 was checked on the same corpora, joined one to four paragraphs at a time with `\n` (14,115 texts), at 6 sizes and 8 widths, against Flutter laid out with the same options:

Option	Layouts	Identical breaks	Different	Heights equal	Deferred
hard breaks	677,520	657,974	**0**	657,974 / 657,974	2.9%
letterSpacing 0.5	677,520	655,457	**0**	655,457 / 655,457	3.3%
letterSpacing -0.3	677,520	646,691	**0**	646,691 / 646,691	4.6%
wordSpacing 2	677,520	657,789	**0**	657,789 / 657,789	2.9%
height 1.3	677,520	657,974	**0**	657,974 / 657,974	2.9%
maxLines 3	677,520	657,974	**0**	657,974 / 657,974	2.9%
combined	677,520	641,769	**0**	641,769 / 641,769	5.3%

Table 4. macOS release build. "Combined" sets letter spacing 0.7, word spacing 1.5, height 1.2 and `maxLines` 4 together.

5.4 · Other scripts and directions

Right-to-left and East Asian text use fonts the platform already has, so nothing is redistributed: a sidecar is built through Flutter for each and checked on 4,000 UI strings per language, from macOS localizations, at 3 sizes and 6 widths, alone, joined with hard breaks, and with spacing.

Text · font	Layouts	Identical	Different	Decided
Hebrew, right-to-left paragraphs · Arial	63,000	31,524	**0**	50%
Hebrew + Latin and digits, mixed direction · Arial	90,000	76,051	**0**	85%
Hebrew, right-to-left · Times New Roman	63,000	31,549	**0**	50%
Hebrew + Latin and digits, mixed · Times New Roman	90,000	76,177	**0**	85%
Arabic (joining forms), right-to-left · Arial	63,000	11,393	**0**	18%
Arabic + Latin and digits, mixed · Arial	90,000	26,634	**0**	30%
Arabic (joining forms) · Times New Roman	63,000	13,964	**0**	22%
Arabic + Latin and digits, mixed · Times New Roman	90,000	30,963	**0**	34%
Japanese and Chinese · Hiragino Sans GB	503,982	308,916	**0**	61%
Korean and Japanese · Apple SD Gothic Neo	503,982	307,145	**0**	61%

Table 5. "Decided": the share of layouts the breaker computes itself; the rest (vowel marks, characters a font lacks, words wider than a narrow line, and all letter-spaced Arabic) go to Flutter. Hiragino Sans GB has no pair positioning (28,092 characters, no pair table); Apple SD Gothic Neo's 16,644 characters need 57×51 pair classes, a 357 KB sidecar.

5.5 · Beyond one plain style

Each extension in §3 and Table 1 has its own check against Flutter laid out with the same text, style and fonts (`flutter_bench/test/`, all run by `tool/validate_all.sh`). The texts are the UI corpora, decomposed into NFD for combining marks, with random points or harakat added for Hebrew and Arabic, cut into spans for rich text, and mixed with Japanese, Chinese, Korean, emoji or Indic words for fallback. Table 6 sums them: 2,498,638 layouts decided, 0 different.

Check · font	Layouts	Identical	Different	Heights equal	Decided
Struts and TextHeightBehavior, 4 settings · Roboto	1,020,480	990,796	**0**	990,796 / 990,796	97%
Font features: tnum, smcp, c2sc, onum, ss01, dlig, frac, liga off · Roboto	680,400	615,540	**0**	615,540 / 615,540	90%
Rich text: 1–4 spans, 4 fonts, 3 sizes, height multipliers, colour changes	36,000	34,458	**0**	34,458 / 34,458	96%
Bundled fallback fonts: Latin-script UI text with Japanese, Chinese, Korean	24,000	20,438	**0**	20,438 / 20,438	85%
Combining marks (NFD French, German, Greek, Russian, Ukrainian) · Roboto	238,824	101,214	**0**	101,214 / 101,214	42%
Combining marks (NFD French, German, Greek, Russian, Ukrainian) · Arial	238,824	153,432	**0**	153,432 / 153,432	64%
Hebrew with points · Arial	101,988	10,722	**0**	10,722 / 10,722	11%
Hebrew with points · Times New Roman	101,988	10,748	**0**	10,748 / 10,748	11%
Arabic with harakat · Arial	101,988	603	**0**	603 / 603	1%
Arabic with harakat · Times New Roman	101,988	1,235	**0**	1,235 / 1,235	1%
Letter-spaced Arabic · Arial	54,000	136	**0**	136 / 136	0%
Letter-spaced Arabic · Times New Roman	54,000	140	**0**	140 / 140	0%
Emoji from the device's font (ZWJ sequences, skin tones, flags)	28,800	15,153	**0**	15,153 / 15,153	53%
Hindi, Bengali, Tamil from the device's fonts, alone and in English	86,112	84,094	**0**	84,094 / 84,094	98%
Thai, Lao, Khmer, Burmese (ICU dictionaries) from the device's fonts	484,038	459,929	**0**	459,929 / 459,929	95%

Table 6. Struts, features and marks on macOS under `flutter_tester`; fonts are the bundled Roboto and the platform's own. Emoji, Indic and the other scripts from the device's fonts are measured once per run on the device (§4). "Decided" counts the layouts the breaker computes; the rest go to Flutter (a character no font here has, a mark left uncomposed next to an excluded character, a word wider than the line).

5.6 · Speed

All methods run on the same paragraphs, the ones break-first decides itself, at 16 px. Each time is the median of 5 passes inside one run of a macOS release build, and then the median of 3 such runs; the speedups below differ by at most 6% between runs.

- **Paragraph heights:** break-first is **9.7×** (docs) and **13.6×** (chat) faster than `TextPainter`.
- **Re-wrapping at a new width** is **8.2×** (docs) and **7.4×** (chat) faster than Flutter's re-layout, which already skips reshaping.
- **A warm word cache** sums cached word widths, which is faster still (15 ms against 23 ms on docs, 32 against 41 on chat). But it is warm only after measuring every word once (the cold column, slower than `TextPainter` on docs), it holds only for the font size and spacing it measured, and it still breaks 3 docs and 3 chat paragraphs differently from Flutter; for other fonts it is wrong far more often (§5.2).

- **End to end**, including paragraphs handed back to Flutter, the gain is 9.6× (docs) and 13.0× (chat).
- **Android emulator**, release, one run: 16.3× (docs) and 23.3× (chat).
- **On GitHub's machines** (median of 3 runs; one on Android): 10.4× / 15.0× on a virtual M1, 9.4× / 12.0× on Linux, and 31.8× / 20.0× on the Android emulator (docs / chat).
- **Native code** is faster still. A line-for-line C port of the default style, called through `dart:ffi`, gives the same line starts in all 2,552,760 checks and is 3.5× (docs) and 4.0× (chat) faster than the Dart breaker, 3.7× end to end. The package stays in Dart, which runs on the web and in any isolate with no native build per platform; the port is a measured option, not a requirement.

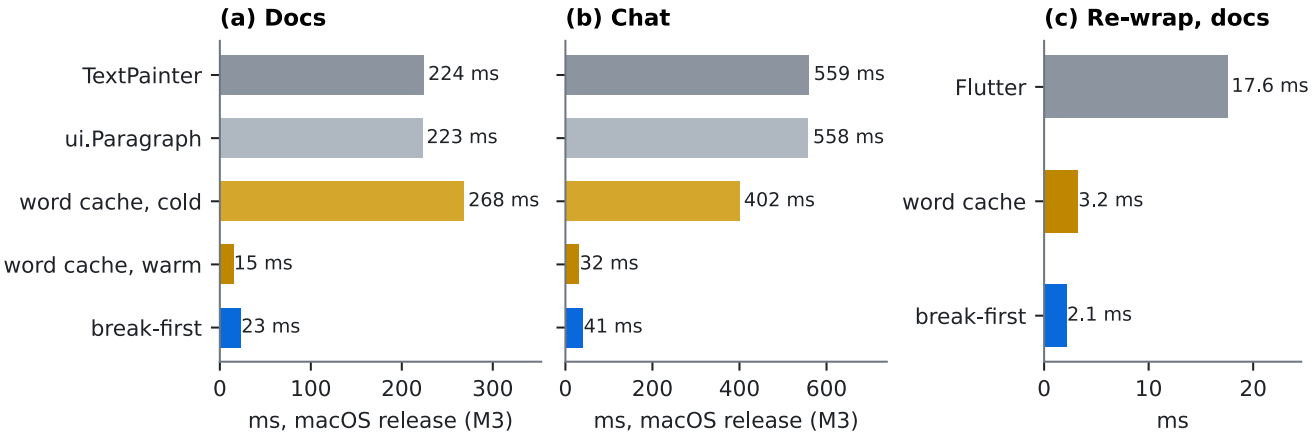


Figure 7. Time to compute every paragraph's height. (a) 5,089 Python documentation paragraphs at 400 px. (b) 23,736 UI messages at 280 px. (c) Re-wrapping the already laid-out docs paragraphs at a new width.

An app-level case: exact list extents. A chat list needs every item's height to place its scrollbar and jump to a message. For 24,000 messages, break-first computes all heights in 53 ms against 585 ms with `TextPainter` (11.1×), fallbacks included. 24,000 of 24,000 heights are equal, and the largest scroll-offset error over the whole list is 0.0 px. On the Android emulator the gain is 17.5×, again with 24,000 equal heights.

The package's example app shows what this buys (video in the repository). A stock `ListView` asked to jump to message #2,500 of 10,000 can only estimate where it is: it lands on #1,969, in a 570 ms frame spent laying out every message on the way. With break-first extents the list lands on #2,500 in a 17 ms frame, and knows its exact total height. All 10,000 extents equal `TextPainter`'s, which takes 420 ms to compute them (Android emulator, release).

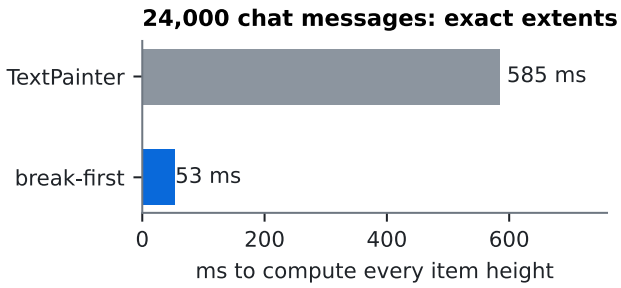


Figure 8. Computing every item's exact height for a chat list.

5.7 · Font sizes and float drift

Flutter's widths are linear in font size, so one file in font units serves every size. But SkParagraph accumulates positions across a paragraph in float32.

- **Exact sizes** (10, 12, 16, 20, 24, 32 px for Roboto): size/units-per-em is a short binary fraction and positions are exact.
- **Other sizes:** positions drift by up to 1.3e-05 of the x position, as much as 0.40 px on a paragraph-long line.

The breaker certifies exact sizes outright. At other sizes it defers any decision that a width error of 2 × 2.5e-05 of the x position could flip: drift at both ends of the line, each allowed 1.9× the largest drift measured. On Android the engine also truncates the font size to 1/64 px, which a single start-up probe detects. With both measures, all 6 sizes in Table 2 had zero mismatches on every platform.

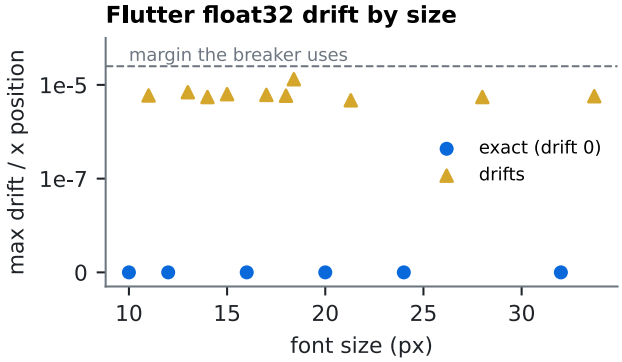


Figure 9. Largest drift between the model and Flutter, relative to x position, by font size.

6 · Where it pays

Break-first does not make drawing cheaper: a line that is painted is still shaped (§8). It pays wherever an app must know how text lays out without drawing it. Table 7 times each such case against what a Flutter app does today, and checks every answer against Flutter's (macOS release build, median of 3 runs; the speedups vary by at most 8% between runs).

Use case	Flutter today	Break-first	Faster	Same answers
Exact extents for a 24,000-message chat list	TextPainter: 585 ms	53 ms	**11.1×**	24,000 / 24,000 heights
Re-wrap 5,089 laid-out paragraphs at a new width (resize, rotation)	re-layout: 18 ms	2.1 ms	**8.2×**	Table 2
Text scale 16 → 20.8 px: every height again (24,000 messages)	TextPainter: 571 ms	16 ms	**36.4×**	24,000 / 24,000 heights
Auto-fit: the largest of 41 sizes that keeps a message in 3 lines (3,000 messages)	TextPainter: 425 ms	11 ms	**40.2×**	3,000 / 3,000 sizes
Paginate 6,455 paragraphs into 994 pages	TextPainter: 428 ms	48 ms	**8.9×**	every page break
Re-paginate at 18 px (1,230 pages)	TextPainter: 440 ms	11 ms	**40.4×**	every page break
Every height in a background isolate (24,000 messages)	not possible	91 ms off the UI isolate, 4.9 ms on it	–	24,000 / 24,000 heights
Heights on a server, no Flutter (24,000 messages)	not possible	40 ms, native executable	–	23,878 / 23,878 decided
Flutter web (CanvasKit): chat-list extents	TextPainter: 1,134 ms	82 ms	**13.9×**	24,000 / 24,000 heights
Flutter web (CanvasKit): auto-fit	TextPainter: 687 ms	17 ms	**39.7×**	3,000 / 3,000 sizes
Flutter web (skwasm): chat-list extents	TextPainter: 976 ms	67 ms	**14.7×**	24,000 / 24,000 heights
Flutter web (skwasm): auto-fit	TextPainter: 762 ms	16 ms	**46.9×**	3,000 / 3,000 sizes

Table 7. Break-first times include the text it hands back to Flutter. A paragraph's records depend on neither the width nor the font size, so a resize, a new text scale, a new page size and each auto-fit probe re-run only the greedy pass. Keeping records costs memory: 14.0 MB for the 24,000 messages, 10.2 bytes per UTF-16 code unit. In a background isolate, Flutter's own layout fails ("UI actions are only available on root isolate"); the breaker computes the heights there, and the UI isolate lays out only the 122 messages it hands back. The server row is `tool/heights_cli.dart` compiled with `dart compile exe`: no Flutter engine, and 122 messages left to the client. Web rows: Chrome, release build.

Who needs it.

- **Long lists and grids of variable height** (chats, feeds, logs, search results, masonry grids): exact extents give an exact scrollbar and an exact jump to any item (§5.6), computed before the items are built, on any isolate, or on the server that sends them.
- **Layouts that follow the window or the user:** resizing on desktop and the web, rotation, split screen and foldables, and the text scale chosen for accessibility.
- **Text fitted to a box:** auto-sizing titles and labels, and the "does it fit in *n* lines" test behind them and behind "read more".
- **Pages:** e-readers, print previews, PDF and report generation, re-paginated when the reader picks a size.

Flutter web. On the web, Flutter lays text out in CanvasKit and skwasm (WebAssembly); the breaker compiles to JavaScript or WebAssembly with the app. The same sidecar, calibrated in the browser (font sizes truncated to 1/64 px, no kerning across a hard break, as on Linux and Android), gives Flutter's line breaks and heights in 47,946 of 47,946 decided layouts (the nine corpora, 300 paragraphs each, 3 sizes × 6 widths; 0 different).

Where it does not pay. Text laid out once and drawn at once, such as a label on screen, gains nothing: Flutter shapes it to paint it anyway. And each font the app ships needs a sidecar, built in 4 s for Roboto and 12 KB gzipped.

7 · Related work

- **TeX's TFM lig/kern programs** are a hand-made per-font width automaton: the closest ancestor of the model, without contextual forms or a certificate. `otftotfm` and LuaTeX's base mode compile an OpenType font's simple ligatures and pair kerns into such programs, without checking them against a shaper.
- **Engines already read shaping tables**, to answer one yes/no question per font or run. Firefox (since 2013) and Blink check whether the space glyph takes part in any lookup, which decides whether shaping word by word is safe. WPF and DirectWrite (`GetTextComplexity`) check whether any default lookup touches a range of characters, which decides whether nominal advances can be used without shaping.

- **PDF libraries** sum AFM widths and kerning pairs, ignoring substitution.
- **WebKit's simple text path** skips the shaper only when kerning and ligatures are off.
- **Android's Minikin and Pretext** measure and cache words, which loses context across words (§5.2). uWrap (2025) predicts line counts for list virtualization from canvas measurements, without claiming exactness.
- **HarfBuzz's** `UNSAFE_TO_CONCAT` makes reshaping after breaking cheaper, but keeps the shape-first order.
- `typeset` (2026) computes widths from advances and pair kerning with a HarfBuzz fallback, without contextual terms or a proof.

We know of no prior work that classifies characters by their role in a font's lookups, proves that shaped width splits into local terms (AAT state machines included), or checks a model made before shaping against a production engine line by line. This survey is a web search, not an exhaustive review.

---

8 · Limitations and threats to validity

Wherever the breaker decides, it is exact (Tables 2–7). What follows is what a reader should doubt, what remains outside a package's reach or needs something this work did not have, and what is handed to Flutter rather than decided.

- **Exact against what.** The theorem is about HarfBuzz's behaviour as the analyzer models it (AAT as HarfBuzz runs it, not as CoreText does). Agreement with Flutter is measured, not proved: every layout in §5 and §6, none different. Measurement has caught the breaker once since the conventions of Table 1 were complete: GitHub's Linux runner, whose Greek strings are not ours, found 2 of 636,882 letter-spaced layouts different (run 41953508573), each a word less than 1/100 px wider than the box (Table 1). The breaker now defers those, the package test checks 42 such words, and every number in this paper was measured again after the fix. On the fix (run 41971344191) the same runner's layouts are 636,568 identical and none different; 314 more go to Flutter than before. Text unlike the corpora may hold conventions no corpus has shown yet.
- **A stale sidecar.** If the font changes and its sidecar is not rebuilt, the canary refuses a wholesale change: with every advance 1% wider it is off by 3.5 px, and used anyway the stale sidecar would break 1,419 of 17,860 layouts. One glyph changed (é 60 units wider) passes the canary and would break 37; the sidecar's digest of its font refuses it when the app passes the font's bytes to `BreakFirst.load`, and rebuilding the sidecar, as the package test does, gives other bytes (`flutter_bench/test/failsafe_test.dart`).
- **Engine updates.** A Flutter app carries its own engine, so a shipped app's shaping cannot change under its sidecar. A Flutter upgrade is a new build, in which the package test rebuilds the sidecar and requires the shipped bytes. Fonts that come from the device (emoji, Indic) are never distilled: they are measured on the device.
- **Baselines.** `TextPainter` is what apps call. A warm word cache is as fast as break-first but not exact (§5.2, §5.6), and needs a first pass that measures every word.
- **Phones.** Mobile timings and checks come from simulators and emulators. The benchmark app runs on a phone with one command (`tool/run_android.sh`), but no phone was available.
- **Painting.** The gain is for measurement and line breaking. Lines that are drawn are still shaped, because Flutter draws text only through `ui.Paragraph`; drawing from the model would need an engine API for positioned glyph runs.
- **Proof.** The certificate rests on the trust base in §3, and its AAT emulation is tested against HarfBuzz, not machine-checked. Fonts with only the pre-2006 `mort` table are refused: no font on the test machine has one to check an analysis against.
- **Handed to Flutter.** Characters that no font here or on the device covers; emoji built on symbols of the Basic Multilingual Plane (❤️, ✍️), whose line-break classes vary; Indic, Thai, Lao, Khmer and Burmese set in a bundled font (they are decided when they come from the device's fonts); letter-spaced Arabic (with ligatures off, lam-alef is still a ligature, and the tables for letter-spaced text have no islands); a combining mark left uncomposed next to a character a rule excludes, and marks in AAT fonts. Emoji and the scripts from the device's fonts are measured once per run through Flutter, not distilled.

---

9 · Reproducibility

All code and results are in the repository (github.com/Rahiche/break-before-you-shape):

- `certify.py`: the analyzer; `census.py`: the census; `bfl.c`: the C prototype and checker;
- `packages/break_first/`: the Flutter package (breaker, rich text, sidecar build step, tests, list example);
- `flutter_bench/`: the benchmark app, the validation and timing harness, and one test per check in Tables 3–6; `flutter_bench/tool/validate_all.sh` runs every local check;
- `make_corpora.py` and `make_test_texts.py`: the corpora and the texts derived from them, rebuilt from the machine's own files rather than shipped;
- `flutter_bench/lib/bench.dart` (`BF_MODE=usecases`), `lib/web_main.dart`, `tool/heights_cli.dart`, `test/use_cases_test.dart` and `test/failsafe_test.dart`: §6 and the stale-sidecar check;
- `native/`: the C port; `.github/workflows/`: the CI that reruns the benchmarks;
- `paper/make_figures.py`: the figures and every number in this paper, read from the result files.

Environment: Flutter 3.47.5 (Dart 3.13.4), HarfBuzz 15.0.0, macOS on an Apple M3, iOS 17.5 simulator, an Android API 35 emulator, and GitHub's macOS 15 and Ubuntu 24.04 runners.